

Projekt: Data-Mart-Erstellung in SQL

Erarbeitungs-/Reflexionsphase

Azzeddine Ouchen

Matrikelnummer: *IU14173271*

Buchtausch-App – PostgreSQL

Ziel der Implementierung

- Umsetzung des relationalen Datenmodells der Buchtausch-App in PostgreSQL
- Erstellung der benötigten Tabellen und Beziehungen
- Definition von Primär- und Fremdschlüsseln
- Absicherung der Datenintegrität durch NOT NULL, UNIQUE und CHECK
- Einfügen von Testdaten mit INSERT INTO
- Überprüfung der SQL-Anweisungen in pgAdmin 4

Umsetzung in PostgreSQL

- Datenbank „**buchtausch**“ wurde in PostgreSQL erstellt
- Umsetzung der Entitäten als relationale Tabellen
- Primärschlüssel zur eindeutigen Identifikation der Datensätze
- Fremdschlüssel zur Sicherstellung der referenziellen Integrität
- Beziehungen wurden durch eigene relationale Strukturen umgesetzt
- Testdaten wurden anschließend mit INSERT INTO eingefügt

Tabelle „Benutzer“

- Speicherung der Benutzerdaten der Buchtausch-App
- Primärschlüssel: BenutzerID
- Pflichtfelder: Name und E-Mail
- Die E-Mail-Adresse ist durch UNIQUE eindeutig
- Zusätzlich werden Adresse und Telefonnummer gespeichert

```
5
6  -- 1. Benutzer
7  CREATE TABLE Benutzer (
8      BenutzerID INT PRIMARY KEY,
9      Name VARCHAR(100) NOT NULL,
10     Adresse VARCHAR(255),
11     "E-Mail" VARCHAR(255) NOT NULL UNIQUE,
12     Telefon VARCHAR(30)
13 );
14
```

```
179 INSERT INTO Benutzer
180 (BenutzerID, Name, Adresse, "E-Mail", Telefon)
181 VALUES
182 (1, 'Max Mustermann', 'Hauptstraße 10', 'max@example.de', '01511111111'),
183 (2, 'Anna Schmidt', 'Bahnhofstraße 20', 'anna@example.de', '01522222222')
184 (3, 'Peter Müller', 'Gartenstraße 5', 'peter@example.de', '01533333333');
185
```

benutzer

Columns (5)

benutzerid	integer
name	character varying(100)
adresse	character varying(255)
E-Mail	character varying(255)
telefon	character varying(30)

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	benutzerid [PK] integer	name character varying (100)	adresse character varying (255)	E-Mail character varying (255)	telefon character varying (30)
1	1	Max Mustermann	Hauptstraße 10	max@example.de	01511111111
2	2	Anna Schmidt	Bahnhofstraße 20	anna@example.de	01522222222
3	3	Peter Müller	Gartenstraße 5	peter@example.de	01533333333

Tabelle „Verlag“

- Speicherung der Verlagsinformationen
- Primärschlüssel: VerlagID
- Der Verlagsname ist ein Pflichtfeld
- Die Tabelle wird über VerlagID mit der Tabelle „Buch“ verknüpft

```
14 -- 1. Verlag
15 -- 2. Verlag
16 CREATE TABLE Verlag (
17     VerlagID INT PRIMARY KEY,
18     Name VARCHAR(100) NOT NULL
19 );
```

```
186 INSERT INTO Verlag
187     (VerlagID, Name)
188 VALUES
189     (1, 'Ravensburger'),
190     (2, 'Carlsen');
```

The diagram shows a table named 'verlag' with two columns: 'verlagid' of type integer and 'name' of type character varying(100). The 'verlagid' column is marked with a primary key icon.

verlag	
Columns (2)	
verlagid	integer
name	character varying(100)

The screenshot shows a 'Data Output' window with a table containing two rows of data. The table has columns for 'verlagid' (marked as a primary key) and 'name'.

	verlagid [PK] integer	name character varying (100)
1	1	Ravensburger
2	2	Carlsen

Tabelle „Genre“

- Speicherung der verschiedenen Buchgenres
- Primärschlüssel: GenreID
- Die Genre-Bezeichnung ist ein Pflichtfeld
- Die Bezeichnung ist durch UNIQUE eindeutig
- Die Tabelle wird über GenreID mit der Tabelle „Buch“ verknüpft

```
20
21  -- 3. Genre
22  CREATE TABLE Genre (
23      GenreID INT PRIMARY KEY,
24      Bezeichnung VARCHAR(100) NOT NULL UNIQUE
25  );
26
```

```
191
192  INSERT INTO Genre
193  (GenreID, Bezeichnung)
194  VALUES
195  (1, 'Roman'),
196  (2, 'Fantasy'),
197  (3, 'Krimi');
198
```

▼ genre

▼ Columns (2)

genreid	integer
bezeichnung	character varying(100)

Data Output Messages Notifications

	genreid [PK] integer	bezeichnung character varying (100)
1	1	Roman
2	2	Fantasy
3	3	Krimi

Tabelle „Sprache“

- Speicherung der verschiedenen Buchsprachen
- Primärschlüssel: **SpracheID**
- Die Sprachbezeichnung ist ein Pflichtfeld
- Die Bezeichnung ist durch UNIQUE eindeutig
- Die Tabelle wird über SpracheID mit der Tabelle „Buch“ verknüpft

```
26
27 -- 4. Sprache
28 CREATE TABLE Sprache (
29     SpracheID INT PRIMARY KEY,
30     Bezeichnung VARCHAR(100) NOT NULL UNIQUE
31 );
32
```

```
199 INSERT INTO Sprache
200     (SpracheID, Bezeichnung)
201     VALUES
202     (1, 'Deutsch'),
203     (2, 'Englisch');
204
```

▼ sprache

▼ Columns (2)

- ▼ spracheid integer
- ▼ bezeichnung character varying(100)

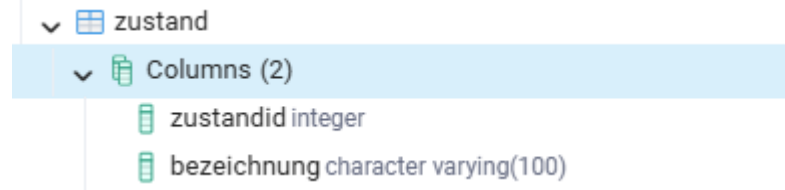
	spracheid [PK] integer	bezeichnung character varying (100)
1	1	Deutsch
2	2	Englisch

Tabelle „Zustand“

- Speicherung des Zustands eines Buches
- Primärschlüssel: **ZustandID**
- Die Zustandsbezeichnung ist ein Pflichtfeld
- Die Bezeichnung ist durch UNIQUE eindeutig
- Die Tabelle wird über ZustandID mit der Tabelle „Buch“ verknüpft

```
33  -- 5. Zustand
34  CREATE TABLE Zustand (
35      ZustandID INT PRIMARY KEY,
36      Bezeichnung VARCHAR(100) NOT NULL UNIQUE
37  );
38
```

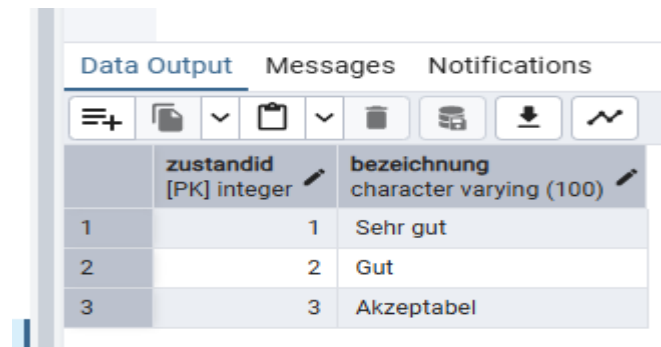
```
204
205  INSERT INTO Zustand
206  (ZustandID, Bezeichnung)
207  VALUES
208  (1, 'Sehr gut'),
209  (2, 'Gut'),
210  (3, 'Akzeptabel');
```



▼ zustand

▼ Columns (2)

- zustandid integer
- bezeichnung character varying(100)



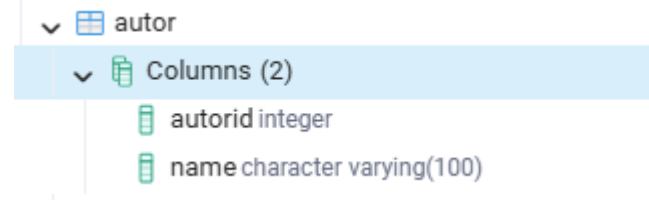
	zustandid [PK] integer	bezeichnung character varying (100)
1	1	Sehr gut
2	2	Gut
3	3	Akzeptabel

Tabelle „Autor“

- Speicherung der Autoreninformationen
- Primärschlüssel: AutorID
- Der Autorenname ist ein Pflichtfeld
- Ein Buch kann über die Zwischentabelle „Buch_Autor“ mit einem oder mehreren Autoren verknüpft werden

```
-- 6. Autor
CREATE TABLE Autor (
  AutorID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL
);
```

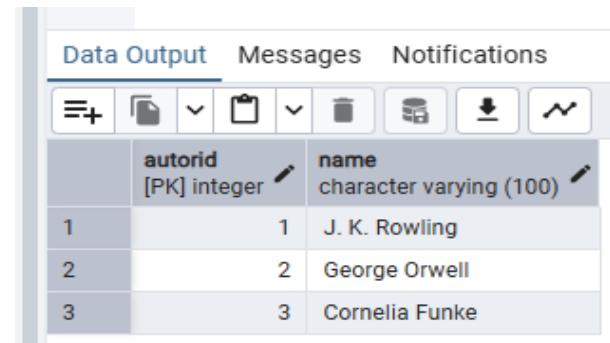
```
211
212 INSERT INTO Autor
213 (AutorID, Name)
214 VALUES
215 (1, 'J. K. Rowling'),
216 (2, 'George Orwell'),
217 (3, 'Cornelia Funke');
```



▼ autor

▼ Columns (2)

- autorid integer
- name character varying(100)



Data Output Messages Notifications

	autorid [PK] integer	name character varying (100)
1	1	J. K. Rowling
2	2	George Orwell
3	3	Cornelia Funke

Tabelle „Abholort“

- Speicherung der Abholorte für die Buchausleihe
- Primärschlüssel: **AbholortID**
- Pflichtfelder: Adresse, PLZ und Ort
- Zusätzlich können Breitengrad und Längengrad gespeichert werden
- Die Tabelle wird über **AbholortID** mit den Tabellen „Angebot“ und „Versand“ verknüpft

```
-- 7. Abholort
45 CREATE TABLE Abholort (
46     AbholortID INT PRIMARY KEY,
47     Adresse VARCHAR(255) NOT NULL,
48     PLZ VARCHAR(10) NOT NULL,
49     Ort VARCHAR(100) NOT NULL,
50     Breitengrad DECIMAL(9,6),
51     Längengrad DECIMAL(9,6)
52 );
```

```
INSERT INTO Abholort
(AbholortID, Adresse, PLZ, Ort, Breitengrad, Längengrad)
VALUES
(1, 'Hauptstraße 10', '44135', 'Dortmund', 51.5136, 7.4653),
(2, 'Bahnhofstraße 20', '44137', 'Dortmund', 51.5030, 7.4570);
```

abholort

Columns (6)

- abholortid integer
- adresse character varying(255)
- plz character varying(10)
- ort character varying(100)
- breitengrad numeric(9,6)
- längengrad numeric(9,6)

Data Output Messages Notifications

Showing rows: 1 to 2 of 1

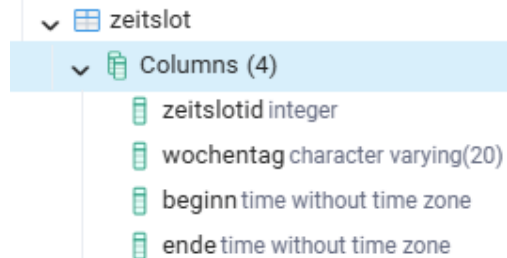
	abholortid [PK] integer	adresse character varying (255)	plz character varying (10)	ort character varying (100)	breitengrad numeric (9,6)	längengrad numeric (9,6)
1	1	Hauptstraße 10	44135	Dortmund	51.513600	7.465300
2	2	Bahnhofstraße 20	44137	Dortmund	51.503000	7.457000

Tabelle „Zeitslot“

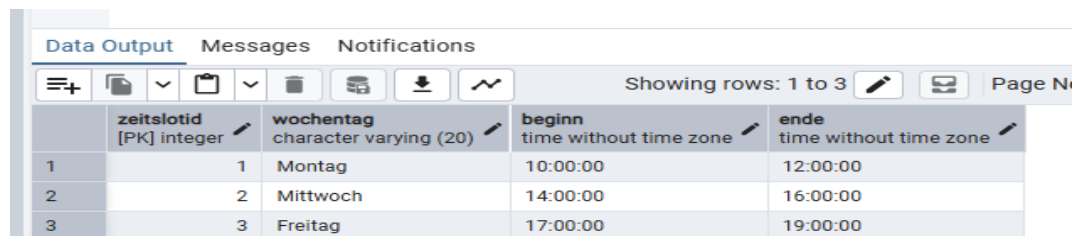
- Speicherung der verfügbaren Zeitfenster für die Buchausleihe
- Primärschlüssel: **ZeitslotID**
- Pflichtfelder: Wochentag, Beginn und Ende
- Mit der CHECK-Bedingung wird sichergestellt, dass **Beginn vor Ende** liegt
- Die Tabelle wird über **ZeitslotID** mit der Tabelle „Ausleihe“ verknüpft

```
55 -- 8. Zeitslot
56 CREATE TABLE Zeitslot (
57     ZeitslotID INT PRIMARY KEY,
58     Wochentag VARCHAR(20) NOT NULL,
59     Beginn TIME NOT NULL,
60     Ende TIME NOT NULL,
61     CHECK (Beginn < Ende)
62 );
63
```

```
225 INSERT INTO Zeitslot
226 (ZeitslotID, Wochentag, Beginn, Ende)
227 VALUES
228 (1, 'Montag', '10:00', '12:00'),
229 (2, 'Mittwoch', '14:00', '16:00'),
230 (3, 'Freitag', '17:00', '19:00');
```



▼	zeitslot
▼	Columns (4)
	zeitslotid integer
	wochentag character varying(20)
	beginn time without time zone
	ende time without time zone



Data Output Messages Notifications				
Showing rows: 1 to 3				
	zeitslotid [PK] integer	wochentag character varying (20)	beginn time without time zone	ende time without time zone
1	1	Montag	10:00:00	12:00:00
2	2	Mittwoch	14:00:00	16:00:00
3	3	Freitag	17:00:00	19:00:00

Tabelle „Versandoption“

- Speicherung der verfügbaren Versandoptionen
- Primärschlüssel: **VersandoptionID**
- Pflichtfeld: **Bezeichnung**
- Die Bezeichnung wird durch **UNIQUE** eindeutig gehalten
- Die Tabelle wird über **VersandoptionID** mit der Tabelle „Versand“ verknüpft

```
64 -- 9. Versandoption
65 CREATE TABLE Versandoption (
66     VersandoptionID INT PRIMARY KEY,
67     Bezeichnung VARCHAR(100) NOT NULL UNIQUE
68 );
69
```

```
231
232 INSERT INTO Versandoption
233 (VersandoptionID, Bezeichnung)
234 VALUES
235 (1, 'Abholung'),
236 (2, 'Postversand');
237
```

▼ versandoption

▼ Columns (2)

- versandoptionid integer
- bezeichnung character varying(100)

	versandoptionid [PK] integer	bezeichnung character varying (100)
1	1	Abholung
2	2	Postversand

Tabelle „Buch“

- Speicherung der Buchinformationen der Buchtausch-App
- Primärschlüssel: **BuchID**
- Pflichtfelder: **Titel, BenutzerID, VerlagID, GenreID, SpracheID und ZustandID**
- Über Fremdschlüssel wird das Buch mit **Benutzer, Verlag, Genre, Sprache und Zustand** verknüpft
- Das Veröffentlichungsjahr ist optional und wird durch eine **CHECK-Bedingung** auf einen positiven Wert begrenzt

```
-- 10. Buch
70
71 CREATE TABLE Buch (
72     BuchID INT PRIMARY KEY,
73     Titel VARCHAR(255) NOT NULL,
74     Veröffentlichungsjahr INT,
75     BenutzerID INT NOT NULL,
76     VerlagID INT NOT NULL,
77     GenreID INT NOT NULL,
78     SpracheID INT NOT NULL,
79     ZustandID INT NOT NULL,
80
81     FOREIGN KEY (BenutzerID) REFERENCES Benutzer(BenutzerID),
82     FOREIGN KEY (VerlagID) REFERENCES Verlag(VerlagID),
83     FOREIGN KEY (GenreID) REFERENCES Genre(GenreID),
84     FOREIGN KEY (SpracheID) REFERENCES Sprache(SpracheID),
85     FOREIGN KEY (ZustandID) REFERENCES Zustand(ZustandID),
86
87     CHECK (Veröffentlichungsjahr IS NULL OR Veröffentlichungsjahr > 0)
88 );
89
```

```
INSERT INTO Buch
(BuchID, Titel, Veröffentlichungsjahr, BenutzerID, VerlagID, GenreID, SpracheID, ZustandID)
VALUES
(1, 'Harry Potter und der Stein der Weisen', 1997, 1, 1, 2, 1, 1),
(2, '1984', 1949, 2, 2, 1, 1, 2),
(3, 'Tintenherz', 2003, 3, 1, 2, 1, 1);
```

buch

Columns (8)

buchid	integer
titel	character varying(255)
veröffentlichungsjahr	integer
benutzerid	integer
verlagid	integer
genreid	integer
spracheid	integer
zustandid	integer

Data Output Messages Notifications

Showing rows: 1 to 3 Page No:

	buchid [PK] integer	titel character varying (255)	veröffentlichungsjahr integer	benutzerid integer	verlagid integer	genreid integer	spracheid integer	zustandid integer
1	1	Harry Potter und der Stein der Wei...	1997	1	1	2	1	1
2	2	1984	1949	2	2	1	1	2
3	3	Tintenherz	2003	3	1	2	1	1

Tabelle „Bewertung“

- Speicherung der Bewertungen zu den Büchern
- Primärschlüssel: **BewertungID**
- Pflichtfelder: **Sterne**, **Bewertungsdatum**, **BenutzerID** und **BuchID**
- Die Anzahl der Sterne wird durch **CHECK** auf Werte von **1 bis 5** begrenzt
- Ein Benutzer kann ein Buch nur einmal bewerten (**UNIQUE**)

```
90 -- 11. Bewertung
91 CREATE TABLE Bewertung (
92     BewertungID INT PRIMARY KEY,
93     Sterne INT NOT NULL,
94     Kommentar TEXT,
95     Bewertungsdatum DATE NOT NULL,
96     BenutzerID INT NOT NULL,
97     BuchID INT NOT NULL,
98
99     FOREIGN KEY (BenutzerID) REFERENCES Benutzer(BenutzerID),
100    FOREIGN KEY (BuchID) REFERENCES Buch(BuchID),
101
102    CHECK (Sterne BETWEEN 1 AND 5),
103    UNIQUE (BenutzerID, BuchID)
104 );
105
```

```
244
245 INSERT INTO Bewertung
246 (BewertungID, Sterne, Kommentar, Bewertungsdatum, BenutzerID, BuchID)
247 VALUES
248 (1, 5, 'Sehr gutes Buch.', '2026-09-10', 2, 1),
249 (2, 4, 'Interessante Geschichte.', '2026-09-11', 3, 2);
250
```

▼ bewertung

▼ Columns (6)

bewertungid	integer
sterne	integer
kommentar	text
bewertungsdatum	date
benutzerid	integer
buchid	integer

Data Output Messages Notifications

	bewertungid [PK] integer	sterne integer	kommentar text	bewertungsdatum date	benutzerid integer	buchid integer
1	1	5	Sehr gutes Buch.	2026-09-10	2	1
2	2	4	Interessante Geschich...	2026-09-11	3	2

Tabelle „Buch_Autor“

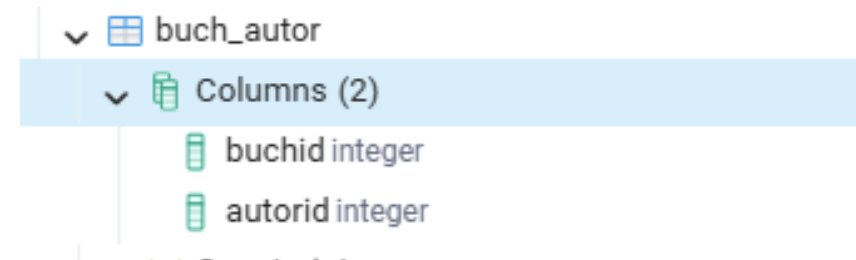
- Verknüpfung zwischen **Buch** und **Autor**
- Umsetzung der **M:N-Beziehung**
- Zusammengesetzter Primärschlüssel: **BuchID und AutorID**
- Beide Attribute sind gleichzeitig **Fremdschlüssel**
- Ein Buch kann mit mehreren Autoren verknüpft werden
- Ein Autor kann mehrere Bücher geschrieben haben

```
-- geschrieben von: Buch <-> Autor (M:N)
CREATE TABLE Buch_Autor (
    BuchID INT NOT NULL,
    AutorID INT NOT NULL,

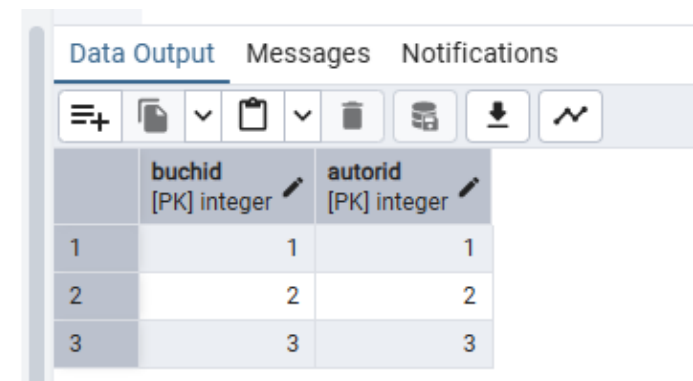
    PRIMARY KEY (BuchID, AutorID),

    FOREIGN KEY (BuchID) REFERENCES Buch(BuchID),
    FOREIGN KEY (AutorID) REFERENCES Autor(AutorID)
);
```

```
250
251 INSERT INTO Buch_Autor
252 (BuchID, AutorID)
253 VALUES
254 (1, 1),
255 (2, 2),
256 (3, 3);
```



buch_autor	
Columns (2)	
buchid	integer
autorid	integer



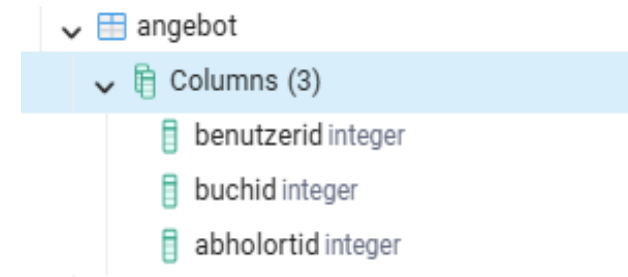
	buchid [PK] integer	autorid [PK] integer
1	1	1
2	2	2
3	3	3

Tabelle „Angebot“

- Speicherung der angebotenen Bücher für den Büchertausch
- Verknüpfung zwischen **Benutzer**, **Buch** und **Abholort**
- Die Tabelle verwendet **BenutzerID**, **BuchID** und **AbholortID** als Fremdschlüssel
- Ein Benutzer kann mehrere Bücher zum Tausch anbieten
- Über **AbholortID** wird der Abholort des Angebots festgelegt

```
121 -- Angebot: Benutzer + Buch + Abholort
122 CREATE TABLE Angebot (
123     BenutzerID INT NOT NULL,
124     BuchID INT NOT NULL,
125     AbholortID INT NOT NULL,
126
127     PRIMARY KEY (BenutzerID, BuchID, AbholortID),
128
129     FOREIGN KEY (BenutzerID) REFERENCES Benutzer(BenutzerID),
130     FOREIGN KEY (BuchID) REFERENCES Buch(BuchID),
131     FOREIGN KEY (AbholortID) REFERENCES Abholort(AbholortID)
132 );
```

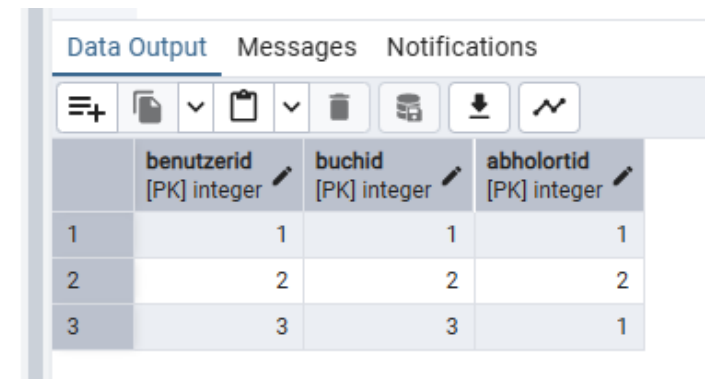
```
257
258 INSERT INTO Angebot
259 (BenutzerID, BuchID, AbholortID)
260 VALUES
261 (1, 1, 1),
262 (2, 2, 2),
263 (3, 3, 1);
264
```



▼  anbot

▼  Columns (3)

-  benutzerid integer
-  buchid integer
-  abholortid integer



Data Output Messages Notifications

	benutzerid [PK] integer	buchid [PK] integer	abholortid [PK] integer
1	1	1	1
2	2	2	2
3	3	3	1

Tabelle „Ausleihe“

- Speicherung der Ausleihen von Büchern
- Verknüpfung zwischen **Benutzer**, **Buch** und **Zeitslot**
- Zusammengesetzter Primärschlüssel: **BenutzerID**, **BuchID** und **Ausleihdatum**
- **BenutzerID**, **BuchID** und **ZeitslotID** sind Fremdschlüssel
- **Ausleihdatum** ist ein Pflichtfeld
- Das **Rueckgabedatum** ist optional
- Eine **CHECK-Bedingung** stellt sicher, dass das Rückgabedatum nicht vor dem Ausleihdatum liegt

```
134 -- Ausleihe: eigenständige relationale Struktur
135 CREATE TABLE Ausleihe (
136     BenutzerID INT NOT NULL,
137     BuchID INT NOT NULL,
138     ZeitslotID INT NOT NULL,
139     Ausleihdatum DATE NOT NULL,
140     Rueckgabedatum DATE,
141
142     PRIMARY KEY (BenutzerID, BuchID, Ausleihdatum),
143
144     FOREIGN KEY (BenutzerID) REFERENCES Benutzer(BenutzerID),
145     FOREIGN KEY (BuchID) REFERENCES Buch(BuchID),
146     FOREIGN KEY (ZeitslotID) REFERENCES Zeitslot(ZeitslotID),
147
148     CHECK (
149         Rueckgabedatum IS NULL
150         OR Rueckgabedatum >= Ausleihdatum
151     )
152 );
```

```
265 INSERT INTO Ausleihe
266 (BenutzerID, BuchID, ZeitslotID, Ausleihdatum, Rueckgabedatum)
267 VALUES
268 (2, 1, 1, '2026-09-12', '2026-09-15'),
269 (3, 2, 2, '2026-09-13', NULL);
```

ausleihe

Columns (5)

- benutzerid integer
- buchid integer
- zeitslotid integer
- ausleihdatum date
- rueckgabedatum date

	benutzerid [PK] integer	buchid [PK] integer	zeitslotid integer	ausleihdatum [PK] date	rueckgabedatum date
1	2	1	1	2026-09-12	2026-09-15
2	3	2	2	2026-09-13	[null]

Tabelle „Versand“

- Speicherung der Versandvorgänge
- Verknüpfung zwischen **Benutzer**, **Buch**, **Versandoption** und **Abholort**
- **BenutzerID**, **BuchID**, **VersandoptionID** und **AbholortID** sind Fremdschlüssel
- Zusammengesetzter Primärschlüssel aus **BenutzerID**, **BuchID**, **VersandoptionID** und **AbholortID**
- Ein Versandvorgang wird eindeutig durch die Kombination der vier Schlüsselattribute bestimmt

```
154 -- Versand: Benutzer + Buch + Versandoption + Abholort
155 CREATE TABLE Versand (
156     BenutzerID INT NOT NULL,
157     BuchID INT NOT NULL,
158     VersandoptionID INT NOT NULL,
159     AbholortID INT NOT NULL,
160
161     PRIMARY KEY (
162         BenutzerID,
163         BuchID,
164         VersandoptionID,
165         AbholortID
166     ),
167
168     FOREIGN KEY (BenutzerID) REFERENCES Benutzer(BenutzerID),
169     FOREIGN KEY (BuchID) REFERENCES Buch(BuchID),
170     FOREIGN KEY (VersandoptionID)
171         REFERENCES Versandoption(VersandoptionID),
172     FOREIGN KEY (AbholortID)
173         REFERENCES Abholort(AbholortID)
174 );
```

```
271 INSERT INTO Versand
272 (BenutzerID, BuchID, VersandoptionID, AbholortID)
273 VALUES
274 (2, 1, 2, 1),
275 (3, 2, 1, 2);
```

▼ versand

▼ Columns (4)

- benutzerid integer
- buchid integer
- versandoptionid integer
- abholortid integer

Data Output Messages Notifications

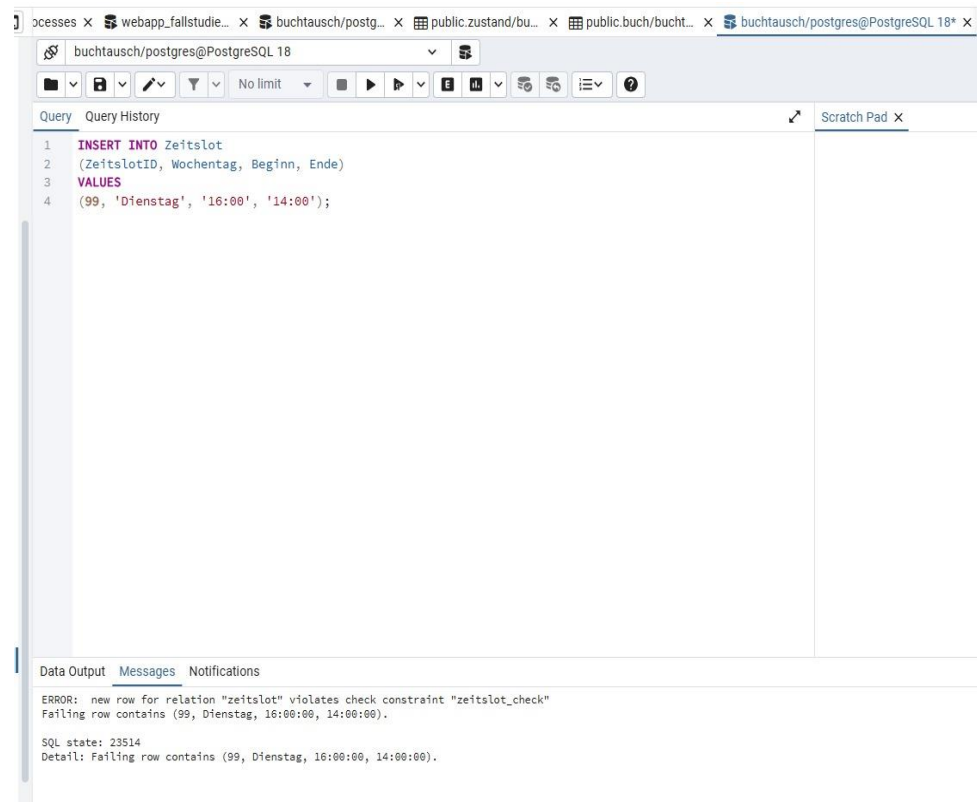
	benutzerid [PK] integer	buchid [PK] integer	versandoptionid [PK] integer	abholortid [PK] integer
1	2	1	2	1
2	3	2	1	2

Erweiterte Datenbanktests und Optimierungen

CHECK-Constraint – Zeitslot

Testfall: Prüfung der CHECK-Constraint Beginn < Ende.

Es wurde ein ungültiger Zeitslot mit einer Endzeit vor der Beginnzeit eingefügt. Die Datenbank hat den Datensatz korrekt abgelehnt.



The screenshot shows a PostgreSQL client window with the following content:

```
1 INSERT INTO Zeitslot
2 (ZeitslotID, Wochentag, Beginn, Ende)
3 VALUES
4 (99, 'Dienstag', '16:00', '14:00');
```

The bottom panel displays the error message:

```
ERROR: new row for relation "zeitslot" violates check constraint "zeitslot_check"
Failing row contains (99, Dienstag, 16:00:00, 14:00:00).

SQL state: 23514
Detail: Failing row contains (99, Dienstag, 16:00:00, 14:00:00).
```

Testdaten – Benutzer

Testfall: Einfügen eines gültigen Benutzerdatensatzes.

Der Datensatz wurde erfolgreich in die Tabelle Benutzer eingefügt.

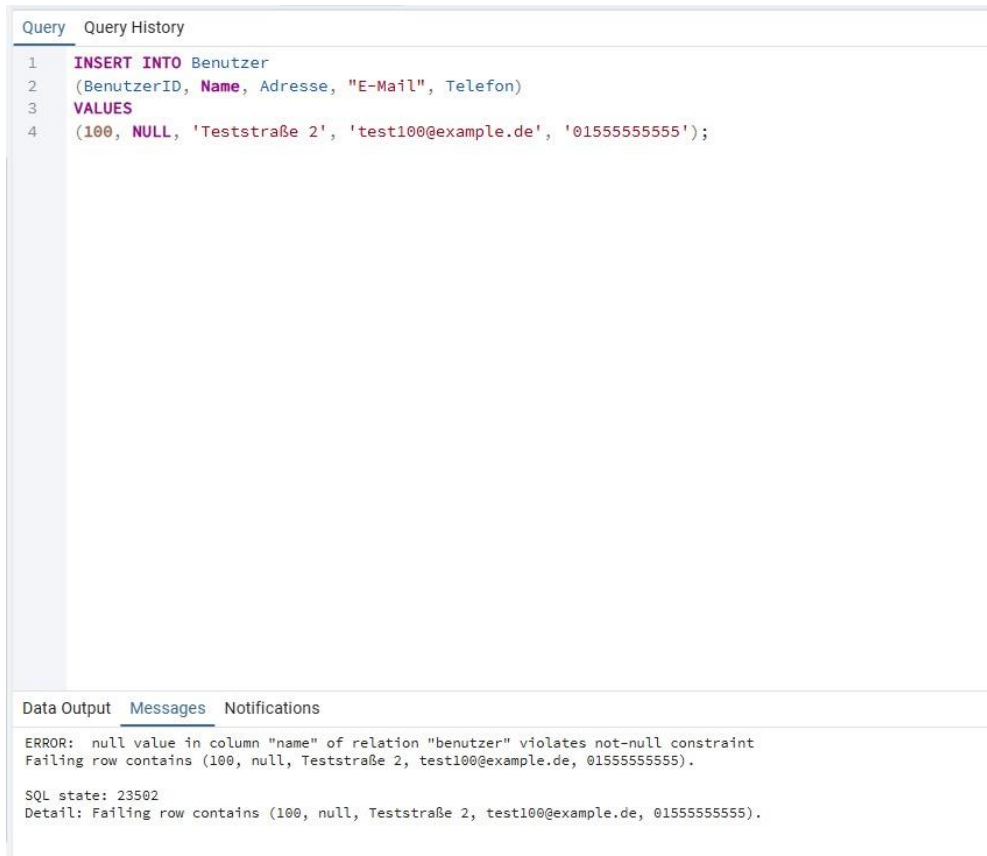
Query	Query History
1	INSERT INTO Benutzer
2	(BenutzerID, Name , Adresse, "E-Mail", Telefon)
3	VALUES
4	(99, 'Test Benutzer', 'Teststraße 1', 'test99@example.de', '01544444444');

Data Output	Messages	Notifications
INSERT 0 1		
Query returned successfully in 79 msec.		

Negative Test – NOT NULL

Testfall: Ein Benutzer wird ohne Namen eingefügt.

Ergebnis: Der Datensatz wird aufgrund der NOT-NULL-Constraint korrekt abgelehnt.



The screenshot shows a database query tool interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL INSERT statement across four lines. The statement is: `1 INSERT INTO Benutzer`, `2 (BenutzerID, Name, Adresse, "E-Mail", Telefon)`, `3 VALUES`, and `4 (100, NULL, 'Teststraße 2', 'test100@example.de', '0155555555');`. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing an error message: `ERROR: null value in column "name" of relation "benutzer" violates not-null constraint`, followed by `Failing row contains (100, null, Teststraße 2, test100@example.de, 0155555555).`. Below the error message, it shows `SQL state: 23502` and `Detail: Failing row contains (100, null, Teststraße 2, test100@example.de, 0155555555).`

```
1 INSERT INTO Benutzer
2 (BenutzerID, Name, Adresse, "E-Mail", Telefon)
3 VALUES
4 (100, NULL, 'Teststraße 2', 'test100@example.de', '0155555555');
```

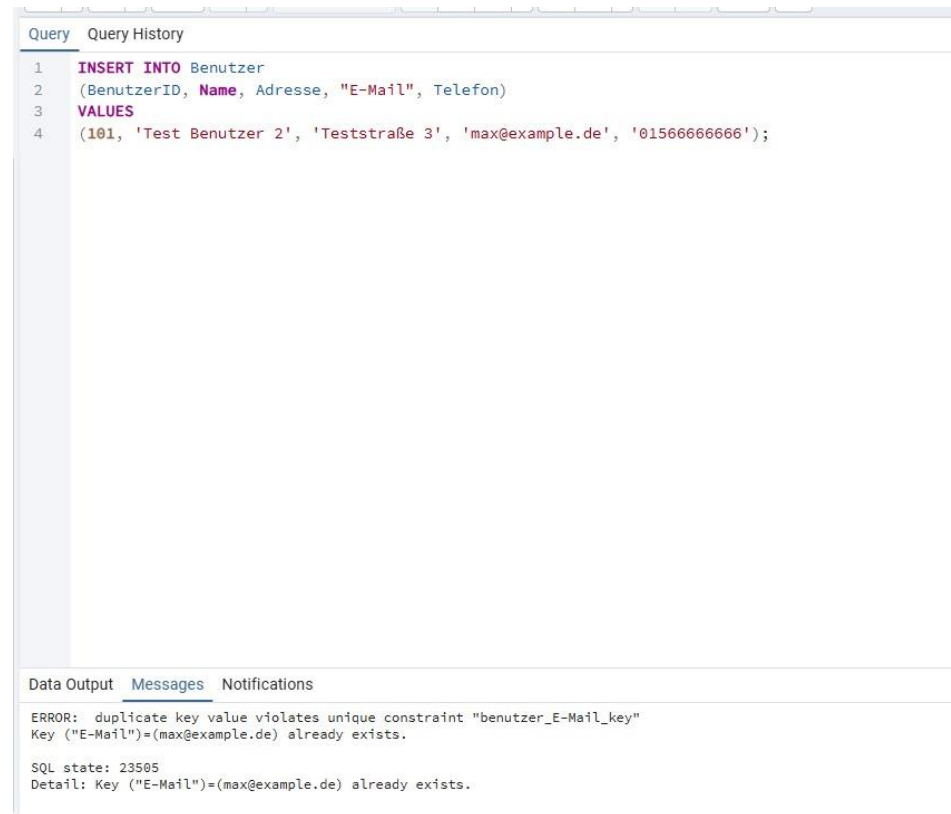
ERROR: null value in column "name" of relation "benutzer" violates not-null constraint
Failing row contains (100, null, Teststraße 2, test100@example.de, 0155555555).

SQL state: 23502
Detail: Failing row contains (100, null, Teststraße 2, test100@example.de, 0155555555).

Negative Test – UNIQUE (E-Mail)

Testfall: Es wird versucht, eine bereits vorhandene E-Mail-Adresse erneut zu verwenden.

Ergebnis: Der Datensatz wird aufgrund der UNIQUE-Constraint korrekt abgelehnt.



```
Query  Query History
1  INSERT INTO Benutzer
2  (BenutzerID, Name, Adresse, "E-Mail", Telefon)
3  VALUES
4  (101, 'Test Benutzer 2', 'Teststraße 3', 'max@example.de', '01566666666');

Data Output  Messages  Notifications
ERROR:  duplicate key value violates unique constraint "benutzer_E-Mail_key"
Key ("E-Mail")=(max@example.de) already exists.

SQL state: 23505
Detail: Key ("E-Mail")=(max@example.de) already exists.
```

Negative Test – Fremdschlüssel

Testfall: Es wird versucht, ein Buch mit einer nicht vorhandenen BenutzerID einzufügen.

Ergebnis: Der Datensatz wird aufgrund der Fremdschlüsselbedingung korrekt abgelehnt.

Query

Query History

1

INSERT INTO Buch

2

(BuchID, Titel, Veröffentlichungsjahr, BenutzerID, VerlagID, GenreID, SpracheID, ZustandID)

3

VALUES

4

(99, 'Test Buch FK', 2026, 999, 1, 1, 1, 1);

Data Output

Messages

Notifications

ERROR: insert or update on table "buch" violates foreign key constraint "buch_benutzerid_fkey"

Key (benutzerid)=(999) is not present in table "benutzer".

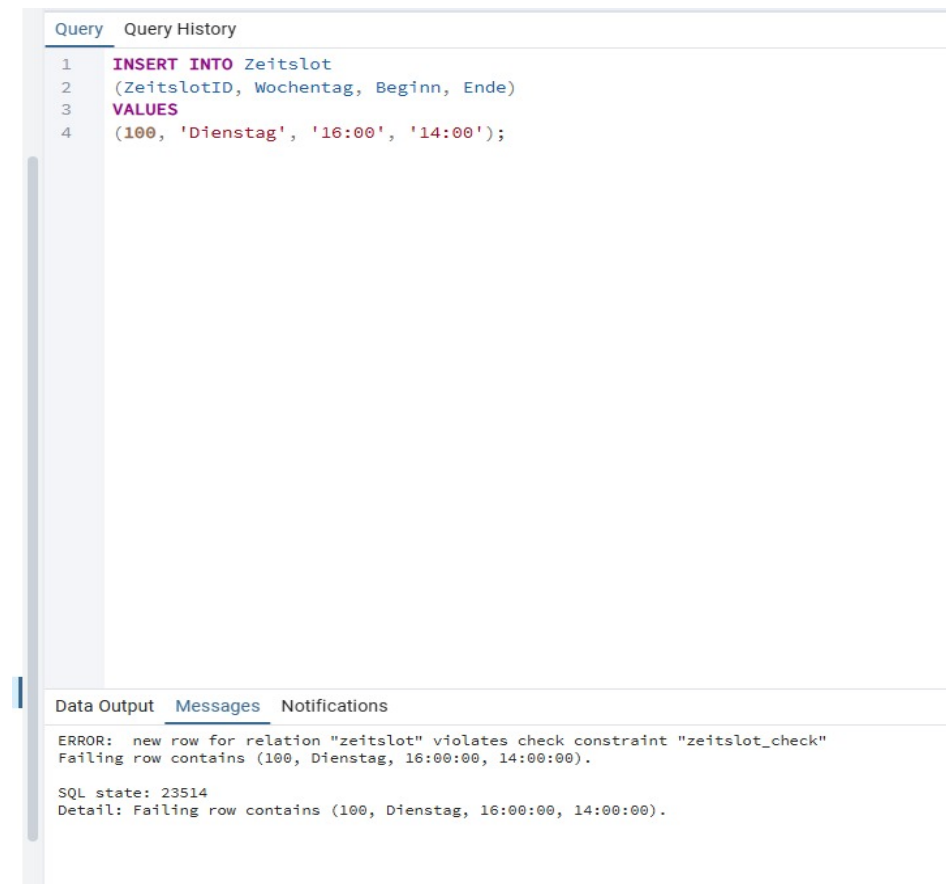
SQL state: 23503

Detail: Key (benutzerid)=(999) is not present in table "benutzer".

Negative Test – CHECK-Constraint (Zeitslot)

Testfall: Es wird ein Zeitslot mit einer Endzeit vor der Beginnzeit eingefügt.

Ergebnis: Der Datensatz wird aufgrund der CHECK-Constraint $\text{Beginn} < \text{Ende}$ korrekt abgelehnt.



The screenshot shows a database query tool interface. The top section, titled 'Query', contains an SQL INSERT statement. The bottom section, titled 'Messages', displays an error message indicating that the insert operation failed due to a check constraint violation.

```
Query History
1  INSERT INTO Zeitslot
2  (ZeitslotID, Wochentag, Beginn, Ende)
3  VALUES
4  (100, 'Dienstag', '16:00', '14:00');
```

Data Output Messages Notifications

ERROR: new row for relation "zeitslot" violates check constraint "zeitslot_check"
Failing row contains (100, Dienstag, 16:00:00, 14:00:00).

SQL state: 23514
Detail: Failing row contains (100, Dienstag, 16:00:00, 14:00:00).

Komplexe JOIN-Abfrage – Buchdetails

Ziel: Zusammenführung von Informationen aus mehreren Tabellen.

Ergebnis: Die Abfrage verbindet Buch, Benutzer, Verlag, Genre, Sprache und Zustand und stellt die Buchinformationen gemeinsam dar.

QueryQuery History

```
1 SELECT
2   b.BuchID,
3   b.Titel,
4   u.Name AS Benutzer,
5   v.Name AS Verlag,
6   g.Bezeichnung AS Genre,
7   s.Bezeichnung AS Sprache,
8   z.Bezeichnung AS Zustand
9 FROM Buch b
10 JOIN Benutzer u ON b.BenutzerID = u.BenutzerID
11 JOIN Verlag v ON b.VerlagID = v.VerlagID
12 JOIN Genre g ON b.GenreID = g.GenreID
13 JOIN Sprache s ON b.SpracheID = s.SpracheID
14 JOIN Zustand z ON b.ZustandID = z.ZustandID;
```

Scratch Pad ✕

Data OutputMessagesNotifications

Showing rows: 1 to 3

Page No: 1 of 1

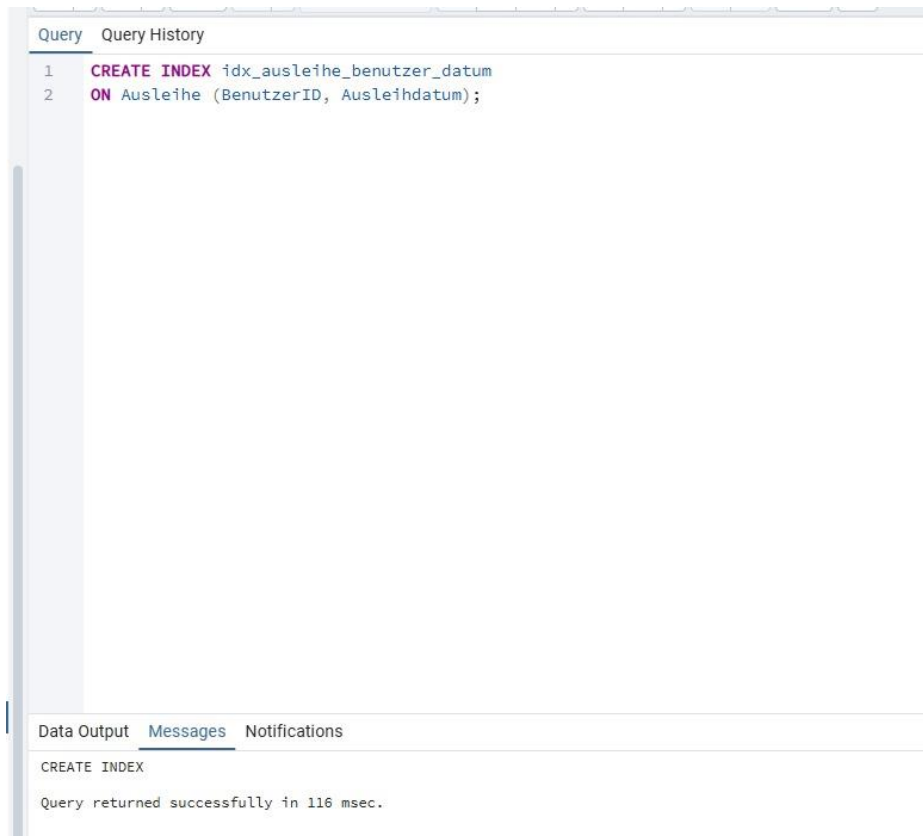
	buchid integer	titel character varying (255)	benutzer character varying (100)	verlag character varying (100)	genre character varying (100)	sprache character varying (100)	zustand character varying (100)
1	3	Tintenherz	Peter Müller	Ravensburger	Fantasy	Deutsch	Sehr gut
2	1	Harry Potter und der Stein der Wei...	Max Mustermann	Ravensburger	Fantasy	Deutsch	Sehr gut
3	2	1984	Anna Schmidt	Carlsen	Roman	Deutsch	Gut

Indexierung – Ausleihe

Ziel: Optimierung häufiger Abfragen auf der Tabelle Ausleihe.

Umsetzung: Ein zusammengesetzter Index auf BenutzerID und Ausleihdatum wurde erstellt.

Ergebnis: Der Index wurde erfolgreich angelegt.



The screenshot shows a SQL IDE interface with a query editor and a messages pane. The query editor contains two lines of SQL code: `1 CREATE INDEX idx_ausleihe_benutzer_datum` and `2 ON Ausleihe (BenutzerID, Ausleihdatum);`. The messages pane at the bottom shows the execution result: `CREATE INDEX` followed by `Query returned successfully in 116 msec.`

```
Query  Query History
1  CREATE INDEX idx_ausleihe_benutzer_datum
2  ON Ausleihe (BenutzerID, Ausleihdatum);
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 116 msec.

JOIN-Abfrage – Ausleihvorgänge

Ziel: Darstellung der Ausleihvorgänge mit den zugehörigen Benutzer-, Buch- und Zeitslot-Informationen.

Umsetzung: Die Tabellen Ausleihe, Benutzer, Buch und Zeitslot werden über JOINS miteinander verknüpft.

Ergebnis: Die Ausleihdaten werden einschließlich Rückgabe, Wochentag und Zeitfenster angezeigt.

QueryQuery HistoryScratch Pad x

```
1 SELECT
2   u.Name AS Benutzer,
3   b.Titel AS Buch,
4   a.Ausleihdatum,
5   a.Rueckgabedatum,
6   z.Wochentag,
7   z.Beginn,
8   z.Ende
9 FROM Ausleihe a
10 JOIN Benutzer u
11     ON a.BenutzerID = u.BenutzerID
12 JOIN Buch b
13     ON a.BuchID = b.BuchID
14 JOIN Zeitslot z
15     ON a.ZeitslotID = z.ZeitslotID
16 ORDER BY a.Ausleihdatum;
```

Data OutputMessagesNotifications

Showing rows: 1 to 2Page No: 1 of 1

	benutzer character varying (100)	buch character varying (255)	ausleihdatum date	rueckgabedatum date	wochentag character varying (20)	beginn time without time zone	ende time without time zone
1	Anna Schmidt	Harry Potter und der Stein der Wei...	2026-09-12	2026-09-15	Montag	10:00:00	12:00:00
2	Peter Müller	1984	2026-09-13	[null]	Mittwoch	14:00:00	16:00:00

Business Rule – Rückgabedatum vor Ausleihdatum

Testfall: Es wird versucht, einen Ausleihvorgang mit einem Rückgabedatum vor dem Ausleihdatum einzufügen.

Ergebnis: Der ungültige Datensatz wird durch die Business Rule korrekt abgelehnt.

Query

Query History

1

INSERT INTO Ausleihe

2

(BenutzerID, BuchID, ZeitslotID, Ausleihdatum, Rueckgabedatum)

3

VALUES

4

(1, 2, 1, '2026-09-20', '2026-09-15');

Data Output

Messages

Notifications

ERROR: new row for relation "ausleihe" violates check constraint "ausleihe_check"

Failing row contains (1, 2, 1, 2026-09-20, 2026-09-15).

SQL state: 23514

Detail: Failing row contains (1, 2, 1, 2026-09-20, 2026-09-15).

Räumliche Suche mit Geokoordinaten

Ziel: Suche nach Abholorten anhand ihrer gespeicherten Geokoordinaten.

Umsetzung: Die Abfrage verwendet Breitengrad und Längengrad, um Abholorte innerhalb eines definierten geografischen Bereichs zu ermitteln.

Ergebnis: Die passenden Abholorte werden zusammen mit ihren Koordinaten angezeigt.

The screenshot shows a database query editor with a query window and a results window. The query window contains an SQL query that selects specific fields from a table named 'Abholort' based on geographic coordinates. The results window displays two rows of data.

Query:

```
1 SELECT
2   AbholortID,
3   Adresse,
4   PLZ,
5   Ort,
6   Breitengrad,
7   Längengrad
8 FROM Abholort
9 WHERE Breitengrad BETWEEN 51.48 AND 51.55
10    AND Längengrad BETWEEN 7.40 AND 7.52;
```

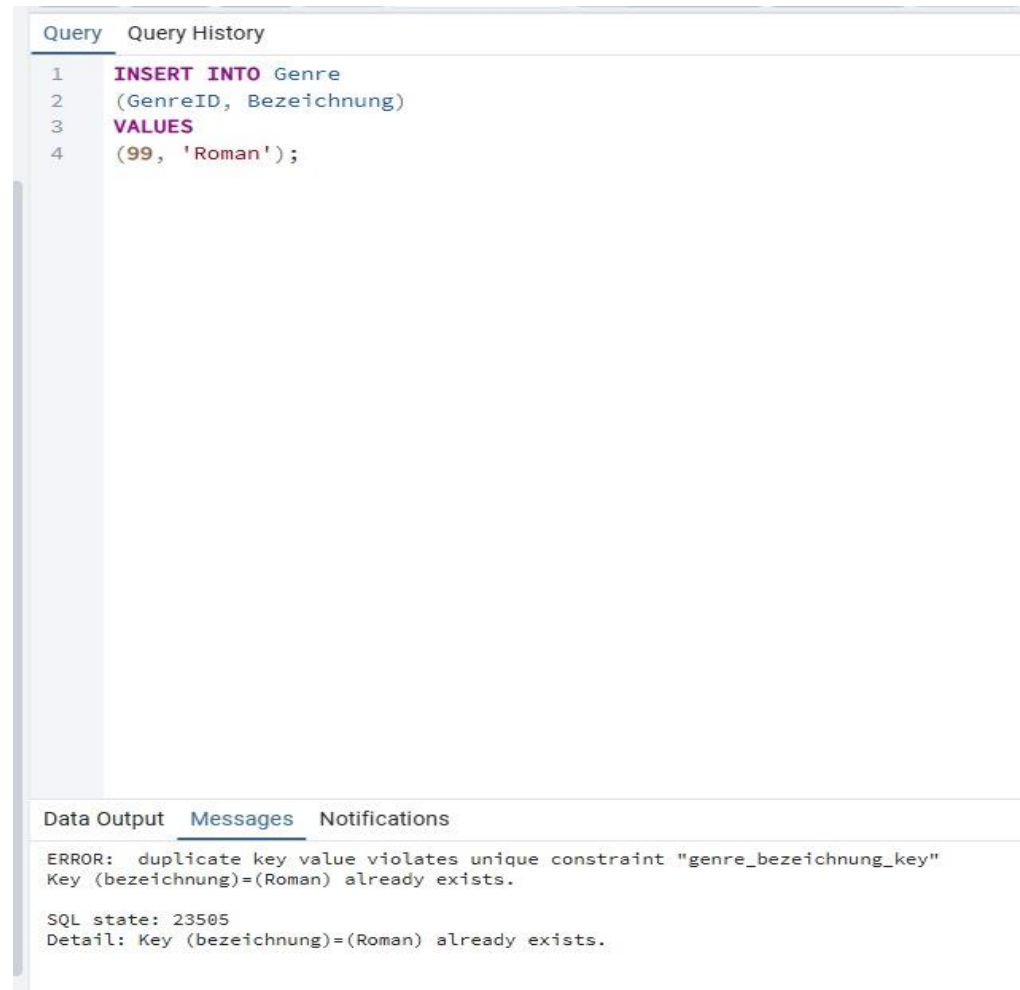
Data Output:

	abholortid [PK] integer	adresse character varying (255)	plz character varying (10)	ort character varying (100)	breitengrad numeric (9,6)	laengengrad numeric (9,6)
1	1	Hauptstraße 10	44135	Dortmund	51.513600	7.465300
2	2	Bahnhofstraße 20	44137	Dortmund	51.503000	7.457000

Negative Test – UNIQUE (Genre)

Testfall: Es wird versucht, eine bereits vorhandene Genre-Bezeichnung erneut einzufügen.

Ergebnis: Der Datensatz wird aufgrund der UNIQUE-Constraint auf Genre.Bezeichnung korrekt abgelehnt.



The screenshot shows a database query tool interface. At the top, there are two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying an SQL INSERT statement across four lines:

```
1  INSERT INTO Genre
2  (GenreID, Bezeichnung)
3  VALUES
4  (99, 'Roman');
```

Below the query editor, there are three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying an error message:

```
ERROR: duplicate key value violates unique constraint "genre_bezeichnung_key"
Key (bezeichnung)=(Roman) already exists.

SQL state: 23505
Detail: Key (bezeichnung)=(Roman) already exists.
```

Negative Test – CHECK (Bewertung)

Testfall: Es wird versucht, eine Bewertung mit 6 Sternen einzufügen, obwohl nur Werte von 1 bis 5 zulässig sind.

Ergebnis: Der Datensatz wird aufgrund der CHECK-Constraint Sterne BETWEEN 1 AND 5 korrekt abgelehnt.

Query

Query History

1

2

3

4

INSERT INTO

Bewertung

(BewertungID, Sterne, Kommentar, Bewertungsdatum, BenutzerID, BuchID)

VALUES

(99, 6, 'Test Bewertung', '2026-09-30', 1, 1);

Data Output

Messages

Notifications

ERROR: new row for relation "bewertung" violates check constraint "bewertung_sterne_check"

Failing row contains (99, 6, Test Bewertung, 2026-09-30, 1, 1).

SQL state: 23514

Detail: Failing row contains (99, 6, Test Bewertung, 2026-09-30, 1, 1).

Komplexe JOIN-Abfrage – Bücher und Bewertungen

Ziel: Zusammenführung von Buch-, Autoren-, Benutzer-, Genre- und Abholortdaten mit den vorhandenen Bewertungen.

Umsetzung: Mehrere Tabellen werden über JOINS verknüpft und mit der Aggregatfunktion AVG() wird die durchschnittliche Bewertung eines Buches berechnet.

Ergebnis: Die Bücher werden mit ihren relevanten Informationen und der durchschnittlichen Sternebewertung dargestellt.

The screenshot shows a database query editor with a query window on the left and a data output window at the bottom. The query is a complex JOIN statement that combines data from several tables: Buch, Autor, Benutzer, Genre, Angebot, Abholort, and Bewertung. It uses the AVG() function to calculate the average rating for each book. The results are displayed in a table with 7 columns: buch, autor, besitzer, genre, abholort, and durchschnittliche_bewertung.

```
1 SELECT
2     b.Titel AS Buch,
3     a.Name AS Autor,
4     u.Name AS Besitzer,
5     g.Bezeichnung AS Genre,
6     ab.Ort AS Abholort,
7     AVG(be.Sterne) AS Durchschnittliche_Bewertung
8 FROM Buch b
9 JOIN Buch_Autor ba
10     ON b.BuchID = ba.BuchID
11 JOIN Autor a
12     ON ba.AutorID = a.AutorID
13 JOIN Benutzer u
14     ON b.BenutzerID = u.BenutzerID
15 JOIN Genre g
16     ON b.GenreID = g.GenreID
17 JOIN Angebot ang
18     ON b.BuchID = ang.BuchID
19 JOIN Abholort ab
20     ON ang.AbholortID = ab.AbholortID
21 LEFT JOIN Bewertung be
22     ON b.BuchID = be.BuchID
23 GROUP BY
24     b.Titel,
25     a.Name,
26     u.Name,
27     g.Bezeichnung,
28     ab.Ort
```

Showing rows: 1 to 3 Page No: 1 of 1

	buch character varying (255)	autor character varying (100)	besitzer character varying (100)	genre character varying (100)	abholort character varying (100)	durchschnittliche_bewertung numeric
1	1984	George Orwell	Anna Schmidt	Roman	Dortmund	4.0000000000000000
2	Harry Potter und der Stein der Wei...	J. K. Rowling	Max Mustermann	Fantasy	Dortmund	5.0000000000000000
3	Tintenherz	Cornelia Funke	Peter Müller	Fantasy	Dortmund	[null]

Negative Test – Fremdschlüssel (Buch → Benutzer)

Testfall: Es wird versucht, ein Buch mit einer nicht vorhandenen BenutzerID einzufügen.

Ergebnis: Der Datensatz wird aufgrund der Fremdschlüsselbeziehung zwischen Buch und Benutzer korrekt abgelehnt.



The screenshot shows a SQL IDE interface with a query editor and a messages pane. The query editor contains an SQL insert statement for a table named 'Buch'. The statement attempts to insert a record with a book ID of 100, title 'Test Foreign Key', publication year 2026, and a user ID of 9999. The user ID 9999 does not exist in the 'benutzer' table, which triggers a foreign key constraint violation. The messages pane at the bottom displays the resulting error: 'ERROR: insert or update on table "buch" violates foreign key constraint "buch_benutzerid_fkey" Key (benutzerid)=(9999) is not present in table "benutzer".' The SQL state is 23503.

```
1 INSERT INTO Buch
2 (BuchID, Titel, Veröffentlichungsjahr, BenutzerID, VerlagID, GenreID, SpracheID, ZustandID)
3 VALUES
4 (100, 'Test Foreign Key', 2026, 9999, 1, 1, 1, 1);
```

ERROR: insert or update on table "buch" violates foreign key constraint "buch_benutzerid_fkey"
Key (benutzerid)=(9999) is not present in table "benutzer".

SQL state: 23503
Detail: Key (benutzerid)=(9999) is not present in table "benutzer".

Indexierung – Buch nach Genre

Ziel: Optimierung von Abfragen nach dem Genre eines Buches.

Umsetzung: Ein Index auf der Spalte GenreID der Tabelle Buch wurde erstellt.

Ergebnis: Der Index wurde erfolgreich angelegt.



The screenshot shows a SQL query editor with two tabs: 'Query' and 'Query History'. The 'Query' tab is active, displaying a two-line SQL statement: `1 CREATE INDEX idx_buch_genre` and `2 ON Buch (GenreID);`. The bottom of the window features three tabs: 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is selected, showing the output: `CREATE INDEX` and `Query returned successfully in 96 msec.`

```
Query  Query History
1  CREATE INDEX idx_buch_genre
2  ON Buch (GenreID);

Data Output  Messages  Notifications
CREATE INDEX
Query returned successfully in 96 msec.
```

Implementierung der Business-Rule-Funktion

Ziel: Umsetzung einer Geschäftsregel für die Ausleihvorgänge.

Regel: Das Rueckgabedatum darf nicht vor dem Ausleihdatum liegen.

Umsetzung: Die Regel wurde als PostgreSQL-Funktion `check_ausleihe_business_rule()` implementiert.

Ergebnis: Die Funktion wurde erfolgreich erstellt.

```
1 CREATE OR REPLACE FUNCTION check_ausleihe_business_rule()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NEW.Rueckgabedatum IS NOT NULL
5         AND NEW.Rueckgabedatum < NEW.Ausleihdatum THEN
6         RAISE EXCEPTION 'Rückgabedatum darf nicht vor dem Ausleihdatum liegen.';
7     END IF;
8
9     RETURN NEW;
10 END;
11 $$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

CREATE FUNCTION

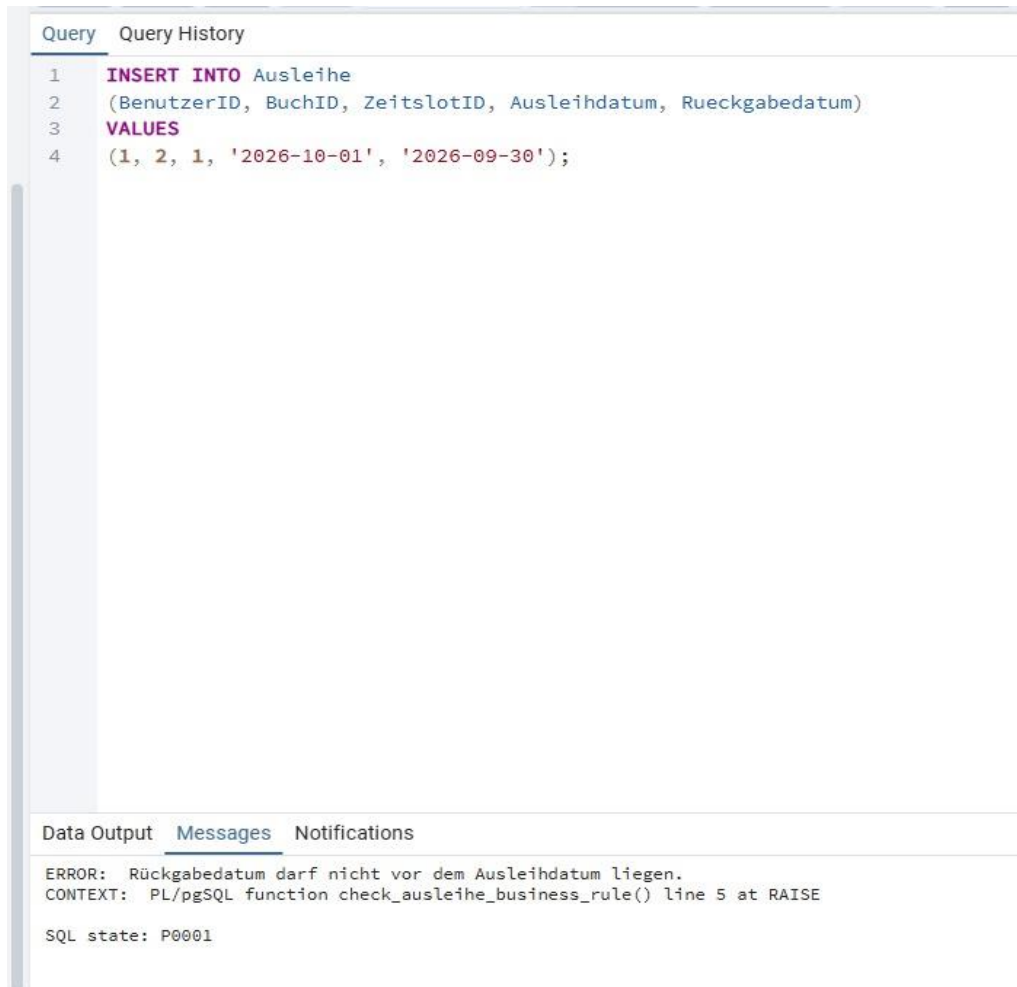
Query returned successfully in 126 msec.

Business-Rule-Test – Ungültiges Rückgabedatum

Testfall: Es wird ein Ausleihvorgang mit einem Rückgabedatum vor dem Ausleihdatum eingefügt.

Erwartetes Ergebnis: Der ungültige Datensatz wird abgelehnt.

Tatsächliches Ergebnis: Die Business Rule löst eine Exception aus und verhindert die Speicherung des Datensatzes.



The screenshot shows a database query editor with a 'Query' tab selected. The query is an SQL INSERT statement into a table named 'Ausleihe'. The statement is as follows:

```
1 INSERT INTO Ausleihe
2 (BenutzerID, BuchID, ZeitslotID, Ausleihdatum, Rueckgabedatum)
3 VALUES
4 (1, 2, 1, '2026-10-01', '2026-09-30');
```

Below the query editor, there is a 'Messages' tab selected, which displays an error message:

```
ERROR: Rückgabedatum darf nicht vor dem Ausleihdatum liegen.
CONTEXT: PL/pgSQL function check_ausleihe_business_rule() line 5 at RAISE

SQL state: P0001
```